KHWAJA YUNUS ALI UNIVERSITY
QUEST FOR KNOWLEDGE

Department of computer science and engineering

Object Oriented Programming II (Java)

Topic: Design and Implementation of a Bank Management System Using Java

Md.Rokonuzzaman | CSE-19th | ID:06225105101036

Table of Contents

1. Introduction
2. Objectives of the Assignment
3. System Overview
4. Implementation of OOP Concepts
    - 4.1 Encapsulation
    - 4.2 Abstraction
    - 4.3 Inheritance
    - 4.4 Polymorphism
    - 4.5 Exception Handling
5. Program Source Code
6. Program Output
7. Output Analysis
8. Conclusion
9. Reflection and AI-Use Statement

1. Introduction

Object-Oriented Programming (OOP) is a programming approach that organizes software around objects and classes. Java is a popular object-oriented programming language that supports important concepts such as encapsulation, abstraction, inheritance, and polymorphism. These concepts help developers create reusable, secure, and maintainable applications.

This assignment presents the design and implementation of a simple Bank Management System using Java. The system manages two types of bank accounts: Savings Account and Current Account. Both accounts share common information, including account number and balance, but they follow different withdrawal policies.

The system uses an abstract parent class named Account and two child classes named SavingsAccount and CurrentAccount. It also implements method overloading, method overriding, runtime polymorphism, and custom exception handling. The main purpose of this assignment is to demonstrate how the fundamental concepts of Java OOP can be applied to a practical banking application.

2. Objectives of the Assignment

The main objectives of this assignment are:

1. To understand the fundamental concepts of Object-Oriented Programming in Java.
2. To implement encapsulation using private data members and getter methods.
3. To understand abstraction through abstract classes and abstract methods.
4. To implement inheritance using parent and child classes.
5. To demonstrate polymorphism using method overloading and overriding.
6. To implement custom exception handling using try, catch, and finally.
7. To develop a simple banking application using Java.

3. System Overview

The Bank Management System is designed to manage two different types of bank accounts. The system contains one abstract parent class and two child classes.

Parent Class: Account

The Account class stores common account information, such as account number and balance. It also provides deposit methods and an abstract withdrawal method.

Child Class 1: SavingsAccount

The SavingsAccount class inherits from Account. It follows a minimum balance policy. A customer must maintain at least 500 in the account after making a withdrawal.

Child Class 2: CurrentAccount

The CurrentAccount class also inherits from Account. It allows customers to withdraw more than their available balance within an overdraft limit of 1000.

Custom Exception: InsufficientBalanceException

This exception is generated when a customer tries to withdraw an invalid amount or violates the account's withdrawal rules.

4. Implementation of Object-Oriented Programming Concepts

4.1 Encapsulation

Encapsulation is the process of combining data and methods into a single class while restricting direct access to the data. In Java, encapsulation is commonly implemented using private variables and public getter and setter methods.

In this program, the Account class contains two private variables: accountNumber and balance. These variables cannot be accessed directly from outside the class. Getter methods are provided to retrieve their values. This approach protects account information and prevents unwanted direct access.

Code Example:

```
class SavingsAccount {

    private String accountNumber;
    private double balance;

    public String getAccountNumber() {
    return accountNumber;
    }

    public double getBalance() {
    return balance;
    }


}
```

4.2 Abstraction

Abstraction means hiding implementation details and showing only the essential features of an object. Java provides abstraction through abstract classes and interfaces.

In this program, Account is declared as an abstract class. It contains an abstract method named withdraw(double amount). The parent class declares the method, but the actual withdrawal implementation is provided by its child classes. Therefore, each account type can follow its own withdrawal rules.

Code Example:

```
    // Abstract Method
    public abstract void withdraw(double amount)
            throws InsufficientBalanceException;
}
```

4.3 Inheritance

Inheritance is a feature of Java that allows one class to acquire the properties and methods of another class. It helps reduce code duplication and improves code reusability.

In this program, SavingsAccount and CurrentAccount inherit from the Account class using the extends keyword. Both child classes receive the common properties and methods of the parent class. They also implement their own withdrawal methods according to their specific requirements.

For example, SavingsAccount maintains a minimum balance of 500, whereas CurrentAccount allows an overdraft limit of 1000.

Code Example:

```
public class SavingsAccount extends Account {

    // Constructor
    public SavingsAccount(String number, double balance) {
        super(number, balance);
    }
```

4.4 Polymorphism

Polymorphism means one method or interface can perform different tasks depending on the object or parameters. Java supports polymorphism through method overloading and method overriding.

Method Overriding:

Method overriding occurs when a child class provides its own implementation of a method inherited from its parent class. In this program, both SavingsAccount and CurrentAccount override the withdraw(double amount) method. Although the method name and parameters are the same, the withdrawal rules are different.

Method Overloading:

Method overloading means creating multiple methods with the same name but different parameters within a class. In this program, the Account class contains two deposit methods. One method accepts a single amount parameter, while the other accepts an amount and a transaction reference.

Runtime Polymorphism:

The program also creates an array of the parent type Account that stores objects of both child classes. A loop calls the withdrawal method on each object. Java automatically executes the appropriate overridden method according to the actual object.

Code Example:

```
// Creating Objects
SavingsAccount savings =
        new SavingsAccount("SA101", 5000);

CurrentAccount current =
        new CurrentAccount("CA101", 2000);

// Method Overloading
System.out.println("\n===== DEPOSIT SECTION =====");

savings.deposit(500);

current.deposit(1000, "REF2026");
```

4.5 Exception Handling

Exception handling is a mechanism in Java that manages runtime errors and prevents abnormal program termination. In this program, a custom exception named InsufficientBalanceException is created by extending the Exception class.

The exception is thrown when a withdrawal does not satisfy the rules of an account. For example, if a savings account tries to withdraw an amount that would reduce its balance below 500, the exception is generated.

The program uses three important keywords: try, catch, and finally. The try block contains the code that may generate an exception. The catch block handles the exception and displays an appropriate error message. The finally block executes whether an exception occurs or not.

Java exceptions are mainly divided into checked and unchecked exceptions. Checked exceptions are checked by the compiler and must be handled or declared. Unchecked exceptions occur at runtime and do not require mandatory handling by the compiler. InsufficientBalanceException is a checked exception because it extends Exception.

Code Example:

```
// Custom Exception Handling
System.out.println("\n===== INVALID TRANSACTION =====");

try {

    savings.withdraw(10000);

} catch (InsufficientBalanceException e) {

    System.out.println(
        "Exception Caught: " + e.getMessage()
    );

} finally {

    System.out.println(
        "Transaction Process Finished."
    );

}
```

5. Program Source Code

The complete Java implementation is provided in the Appendix of this report. It includes the following classes:

1. InsufficientBalanceException
2. Account
3. SavingsAccount
4. CurrentAccount
5. BankManagementSystem

The main method creates account objects, performs deposit and withdrawal operations, demonstrates method overloading and runtime polymorphism, and handles invalid transactions using a custom exception.

```
class InsufficientBalanceException extends Exception {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

abstract class Account {
    private String accountNumber;
    private double balance;

    public Account(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposit successful: " + amount);
    }

    // Method Overloading
    public void deposit(double amount, String reference) {
        balance += amount;
        System.out.println("Deposit: " + amount +
                           "        | Reference: " + reference);
    }

    protected void updateBalance(double amount) {
        balance = amount;
    }

    public abstract void withdraw(double amount)
            throws InsufficientBalanceException;
}

class SavingsAccount extends Account {

    public SavingsAccount(String number, double balance) {
        super(number, balance);
    }

    @Override
    public void withdraw(double amount)
            throws InsufficientBalanceException {

        if (amount <= 0 || getBalance() - amount < 500) {
            throw new InsufficientBalanceException(
                "Savings account must maintain a minimum balance of 500."
            );
        }

        updateBalance(getBalance() - amount);
        System.out.println("Savings withdrawal successful: " + amount);
    }
}

class CurrentAccount extends Account {

    private static final double OVERDRAFT_LIMIT = 1000;

    public CurrentAccount(String number, double balance) {
        super(number, balance);
    }

    @Override
    public void withdraw(double amount)
```

```
            getBalance() - amount < -
OVERDRAFT_LIMIT) {

            throw                  new
InsufficientBalanceException(
                "Current      account
overdraft limit exceeded."
            );
        }

        updateBalance(getBalance()  -
amount);
        System.out.println("Current
withdrawal successful: " + amount);
    }
}

public class BankManagementSystem {

    public static void main(String[]
args) {

        System.out.println("Student
ID: YOUR_STUDENT_ID");
        System.out.println("=====
BANK MANAGEMENT SYSTEM =====");

        SavingsAccount savings =
            new
SavingsAccount("SA101", 5000);

        CurrentAccount current =
            new
CurrentAccount("CA101", 2000);

        savings.deposit(500);
        current.deposit(1000,
"REF2026");

        Account[]     accounts     =
{savings, current};

        for   (Account   account   :
accounts) {


System.out.println("\nAccount Number:
"
            throws
InsufficientBalanceException {

        if (amount <= 0 ||
                } else {

account.withdraw(2500);
                }

            }                    catch
(InsufficientBalanceException e) {


System.out.println("Exception:   "   +
e.getMessage());

            } finally {


System.out.println("Final Balance: "
                        +
account.getBalance());

System.out.println("Transaction
completed.");
            }
        }

        System.out.println("\n=====
INVALID TRANSACTION =====");

        try {
            savings.withdraw(10000);

        }                        catch
(InsufficientBalanceException e) {


System.out.println("Exception caught:
" + e.getMessage());

        } finally {


System.out.println("Transaction
process finished.");
        }
    }
}
```

```
                    +
account.getAccountNumber());

            try {
                if          (account
instanceof SavingsAccount) {

account.withdraw(1000);
```

6. Program Output

The following output is generated after successfully compiling and executing the Java program.

```
BANK MANAGEMENT SYSTEM
==============================

===== DEPOSIT SECTION =====
Deposit successful: 500.0
Deposit: 1000.0
Reference: REF2026

===== WITHDRAWAL SECTION =====

Account Number: SA101
Savings withdrawal successful: 1000.0
Final Balance: 4500.0
Transaction Completed.

Account Number: CA101
Current withdrawal successful: 2500.0
Final Balance: 500.0
Transaction Completed.

===== INVALID TRANSACTION =====
Exception Caught: Savings account must maintain a minimum balance of 500.
Transaction Process Finished.
```

7. Output Analysis

The output shows that the Bank Management System successfully performs deposit and withdrawal operations for both types of accounts. Initially, the savings account has a balance of 5000. After depositing 500 and withdrawing 1000, its final balance becomes 4500. Similarly, the current account starts with 2000. After depositing 1000 and withdrawing 2500, its final balance becomes 500.

The output demonstrates runtime polymorphism because the same withdrawal method is executed differently for SavingsAccount and CurrentAccount objects. The invalid transaction section demonstrates custom exception handling. When the savings account attempts to withdraw 10000, the program generates an InsufficientBalanceException. The catch block handles the exception, and the finally block displays the transaction completion message.

8. Conclusion

In conclusion, this assignment demonstrates the practical implementation of the major Object-Oriented Programming concepts in Java through a Bank Management System. The program uses encapsulation to protect account information, abstraction to hide implementation details, and inheritance to reuse common properties and methods. It also demonstrates polymorphism through method overloading and method overriding. Furthermore, custom exception handling is implemented to manage invalid withdrawal operations.

Through this assignment, it becomes clear how different OOP concepts work together to develop a structured, reusable, and maintainable Java application. The Bank Management System provides a simple example of applying theoretical programming concepts to a real-world problem.

9. Reflection and AI-Use Statement

While completing this assignment, I gained practical knowledge of Java Object-Oriented Programming. I learned how encapsulation protects private data, how abstraction hides unnecessary implementation details, and how inheritance allows code reuse. I also understood the difference between method overloading and method overriding through practical examples. Implementing a custom exception helped me understand how Java handles invalid operations. Overall, this assignment improved my understanding of object-oriented programming and strengthened my Java programming skills.

AI-Use Statement: I used ChatGPT as a learning assistant to understand the assignment requirements, organize the program structure, and review Java OOP concepts. I reviewed the code and explanations to understand how the program works.